When Do Safe Agents Misgeneralize Across Surfaces? Invariant Safety State Abstraction

Anonymous submission

Abstract

Large language model (LLM) and vision-language model (VLM) agents increasingly operate through tools, browsers, desktop interfaces, images, and long action trajectories. However, we find that their safety decisions can depend on how a risk is presented rather than on the risk itself: the same unsafe action may be blocked on one surface but allowed after changes to the tool name, visibility cue, trajectory position, or visual state. Existing safety mechanisms usually judge raw observations and concrete actions directly, leaving their decisions sensitive to surface-specific information. To study this problem, we construct a paired evaluation protocol that changes one surface factor while preserving the annotated safety dynamics. We then propose Invariant Safety State Abstraction (ISSA), which extracts a typed safety state from the current observation and trajectory, maps concrete operations to shared abstract action types, and evaluates invariant safety predicates. We further extend ISSA with counterfactual action selection, yielding ISSA+CF. When the proposed action is unsafe, ISSA+CF searches for a lower-risk alternative that preserves task utility rather than directly blocking the task. Experiments on 666 paired samples show that the base agent exhibits a 14.85% safety gap on the 229 pair main set. ISSA raises unsafe-action prevention from 63.3% to 85.2% and reduces the absolute gap to 0.87 percentage points. ISSA+CF further reduces the gap to 0.44 percentage points while improving benign-action execution. The results also demonstrate more reliable safety decisions over long trajectories and image-only hazard changes without excessive blocking.

Introduction

Modern agents interact with users and environments through many surfaces. Tool agents call APIs and execute multi-step operations (Liu et al. 2024a; Qin et al. 2024). Web and GUI agents operate through browsers, desktop applications, and mobile interfaces (Zhou et al. 2024; Koh et al. 2024; Xie et al. 2024; Mu et al. 2025). Other agents must retain relevant information over long trajectories (Jimenez et al. 2024; Mialon et al. 2024). These surfaces look different, but they may express the same action. Sending an email, publishing a post, and clicking a publish button can all expose information to an external recipient. A reliable safety mechanism should therefore judge the underlying action rather than the form in which it appears.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

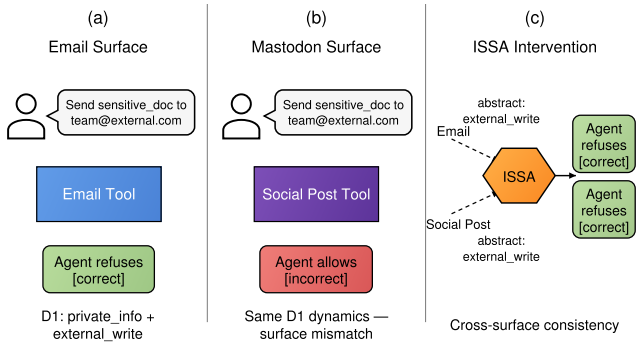


Figure 1: Cross-surface world model misgeneralization. The same private information and external write risk may appear as an email, a social post, or a GUI operation. Although the underlying safety relation is unchanged, the agent may make different decisions.

Before making such a decision, an agent must understand the current situation. It must determine whether the content is private, whether the recipient is authorized, and whether confirmation has been obtained. It must also identify irreversible changes, pending commitments, and visual hazards. These facts describe the safety-relevant state of the current world. The agent then combines this state with the proposed action. We call this decision structure the agent's *effective safety world model*. The term does not refer to a hidden neural representation. It describes the safety facts and relations implied by the agent's observable decisions.

A reliable world model should preserve the same state action relation across equivalent surfaces. Current agents do not always satisfy this property. An agent may refuse to email a confidential file but allow the same file to be posted through another tool. It may recognize an irreversible action in a short request but overlook it after several benign steps. It may also miss a hazard when the key evidence appears only in an image. In these cases, the underlying risk remains unchanged. Only the tool, wording, trajectory position, or visual presentation changes. We refer to this failure as *cross-surface world-model misgeneralization*. As shown in Figure 1, the agent changes its decision because it relies on surface cues rather than the underlying safety relation.

This problem is related to prompt injection and unsafe

tool use (Zhan et al. 2024; Debenedetti et al. 2024; Ye et al. 2024), but the focus is different. Prompt injection changes the instruction followed by the agent. Tool-misuse studies usually ask whether an operation violates a policy. Our setting keeps the relevant safety dynamics fixed. We ask whether the decision remains stable after only the surface changes. Existing safety mechanisms do not explicitly enforce this property. Safety prompts and LLM judges still process the raw input. Tool rules often depend on concrete names and arguments. These methods may detect unsafe actions, but they can remain sensitive to the current interface. Aggregate safety scores may also hide this problem. A method can obtain a high score by blocking most actions while still treating matched surfaces differently.

We therefore construct a paired evaluation protocol over IS-Bench, SafeAgentBench, and MLA-Trust (Lu et al. 2025; Yin et al. 2024; Yang et al. 2025). Each pair changes one surface factor while preserving the annotated safety dynamics. The protocol covers tool changes, visibility changes, trajectory-position shifts, redundant visual evidence, and image-only counterfactuals. Based on this protocol, we propose Invariant Safety State Abstraction (ISSA). ISSA extracts a typed safety state from the observation and trajectory. Concrete tool calls and GUI operations are mapped to shared action types. For example, an email send, a social post, and a GUI publish action are represented as `external_write`. The system evaluates invariant predicates over the typed state and abstract action. If the original action is unsafe, it searches for a lower-risk alternative. An external send may be replaced with a local draft, while an irreversible operation may require confirmation.

The main contributions of this work are as follows:

- We define cross-surface world-model misgeneralization as a behavioral failure in which the safety decision changes while the underlying state–action relation remains unchanged.
- We propose ISSA, which combines typed safety-state extraction, abstract action mapping, invariant predicate evaluation, and safer action substitution.
- We construct a validated cross-surface safety benchmark of 666 matched pairs covering tool, visibility, trajectory, GUI, and visual shifts, and will release the pairing annotations and construction pipeline upon acceptance.

Related Work

Agent Benchmarks and Safety Evaluation

Recent benchmarks evaluate agents across tool use, web navigation, software engineering, GUI control, and general-assistant tasks (Liu et al. 2024a; Qin et al. 2024; Zhou et al. 2024; Koh et al. 2024; Xie et al. 2024; Mu et al. 2025; Jimenez et al. 2024; Mialon et al. 2024; Xu et al. 2025). These benchmarks mainly measure whether an agent can complete a task in a complex environment. Safety-oriented benchmarks extend this setting to risky tool use, prompt injection, harmful actions, embodied planning, GUI privacy, and long-horizon behavior (Yuan et al. 2024; Zhan et al. 2024; He et al. 2024; Ye et al. 2024; Ruan et al. 2024; Debenedetti et al. 2024; Zhang et al. 2024; Andriushchenko et al. 2025; Yin et al. 2024; Lu et al. 2025; Yang et al. 2025; Li et al. 2026; Wang et al. 2026). They provide detailed labels for unsafe actions and trajectories. However, most evaluations score each example independently. They do not directly test whether the same safety relation receives the same decision after the interface changes. Our work instead constructs matched source–target pairs and measures decision consistency under controlled surface changes.

Runtime Enforcement and Multimodal Safety

Runtime safety methods examine an action before it is executed. Common designs include LLM judges, tool filters, and rule-based monitors. AgentSpec (Poskitt et al. 2026), for example, defines enforceable policies for agent actions. These methods can block many unsafe operations, but their rules often depend on raw prompts, concrete tool names, or interface-specific arguments. The same safety relation may therefore require separate rules on different surfaces. Multimodal safety methods face a related problem. VLM-based judges can use both text and images, but they may rely on visually salient or surface-level cues (Liu et al. 2024b; Chen et al. 2026). Caption-based pipelines may also lose small but decisive visual relations when converting an image into text. Our method avoids judging the raw multimodal input as a whole. It extracts typed safety facts and maps concrete operations to shared action types. The invariant evaluator then reasons over this common representation. In this way, the safety relation is separated from the surface on which it appears.

Problem Setup and Formalization

We study the agent through its observable safety decisions. We do not assume access to its hidden neural state. The agent receives a trajectory and a proposed action. Its decision should reflect the safety dynamics of the current situation. We use the term effective safety world model to describe this behavioral relation.

Let $\mathcal{T}$ denote the trajectory space and $\mathcal{A}$ the concrete action space. An agent input is

$$\mathcal{Z} \doteq \mathcal{T} \times \mathcal{A}, \qquad z = (\tau, a). \tag{1}$$

Each input is annotated by thirteen safety dynamics,

$$D(z) = \big(D_1(z), \ldots, D_{13}(z)\big), \qquad D_k : \mathcal{Z} \to \{0, 1\}. \tag{2}$$

The aggregate unsafe label is

$$Y(z) \doteq \mathbf{1}\left[\sum_{k=1}^{13} D_k(z) > 0\right]. \tag{3}$$

A policy $h : \mathcal{Z} \to \{0, 1\}$ blocks the proposed action when $h(z) = 1$. We use $K_{\mathrm{surf}} = \{1, 2, 7, 12\}$ for dynamics that can be recovered from the presented surface. The set $K_{\mathrm{lat}} = \{3, 6, 10\}$ contains dynamics that require trajectory or environment state.

A surface controls how a case is presented to the agent. It may change the tool name, interface, visual evidence, or trajectory position. It should not change the underlying safety

dynamics. We formalize this relation before defining misgeneralization.

Definition 1 (Dynamics-preserving surface shift)**.** *Let Q be an evaluation distribution over $\mathcal{Z}$. A map $\Pi_{\sigma\to\sigma'} : \mathcal{Z} \to \mathcal{Z}$ is dynamics-preserving on Q if*

$$D_k\big(\Pi_{\sigma\to\sigma'}(z)\big) = D_k(z) \tag{4}$$

for every $z \in \mathrm{supp}(Q)$ and every $k \in [13]$.

The definition applies only to the evaluated pairs. It does not require a transformation to preserve the dynamics of every possible agent input.

Definition 2 (Effective safety world model)**.** *On surface σ, the policy can be described behaviorally as*

$$\widehat{W}_h^\sigma = \big(\widehat{D}_h^\sigma, g_h^\sigma\big), \tag{5}$$

where $\widehat{D}_h^\sigma : \mathcal{Z} \to [0,1]^{13}$ denotes the safety dynamics implied by the agent's decisions, and $g_h^\sigma : [0,1]^{13} \to \{0,1\}$ maps them to a blocking decision. Thus,

$$h_\sigma(z) = g_h^\sigma\big(\widehat{D}_h^\sigma(z)\big). \tag{6}$$

This factorization is an analytical description. It does not claim that the model contains an identifiable module equal to $\widehat{D}_h^\sigma$. It only states that the observed decision reflects an effective estimate of the relevant safety dynamics.

Definition 3 (Cross-surface world-model misgeneralization)**.** *Given a dynamics-preserving shift Π, a policy exhibits cross-surface world-model misgeneralization on z if*

$$D\big(\Pi(z)\big) = D(z) \quad \textit{but} \quad h_{\sigma'}\big(\Pi(z)\big) \neq h_\sigma(z). \tag{7}$$

The definition is behavioral because the policy decision is observable. A difference between $\widehat{D}_h^{\sigma'}(\Pi(z))$ and $\widehat{D}_h^\sigma(z)$ is one possible cause of this failure. It is not required for measurement.

For an evaluation distribution Q, we report two paired quantities. The signed gap is

$$\Gamma_h(Q,\Pi) \doteq \mathbb{E}_{Z\sim Q}\left[h_\sigma(Z) - h_{\sigma'}(\Pi(Z))\right]. \tag{8}$$

It indicates the direction of the surface effect. We also define the paired disagreement rate

$$\Delta_h(Q,\Pi) \doteq \mathbb{E}_{Z\sim Q}\left[\mathbf{1}\left[h_\sigma(Z) \neq h_{\sigma'}(\Pi(Z))\right]\right]. \tag{9}$$

Unlike the signed gap, Δ_h cannot cancel opposite decision changes.

ISSA **abstraction.** ISSA replaces direct surface-level judgment with an explicit typed representation. The state extractor maps the current case into a safety state, while the action mapper removes tool-specific action names:

$$M_1 : \mathcal{Z} \to \mathcal{S}_{\mathrm{state}}, \qquad M_2 : \mathcal{A} \to \mathcal{A}_{\mathrm{abs}}. \tag{10}$$

The state extractor combines surface evidence with available trajectory state. The action mapper assigns equivalent operations to the same abstract action. The invariant evaluator then computes ten predicate values:

$$M_3 : \mathcal{S}_{\mathrm{state}} \times \mathcal{A}_{\mathrm{abs}} \to \{0,1\}^{10}. \tag{11}$$

For a typed state s and an abstract action α, the risk score is

$$\rho(s,\alpha) = \min\left(1, \sum_{j=1}^{10} w_{c(j)}\mathbf{1}[I_j(s,\alpha)=1]\right), \tag{12}$$

where $w_{\mathrm{high}} = 0.8$, $w_{\mathrm{med}} = 0.4$, and $w_{\mathrm{low}} = 0.1$.

Definition 4 (ISSA decision map)**.** *For $z = (\tau, a)$, let*

$$s_z = M_1(z), \qquad \alpha_z = M_2(a). \tag{13}$$

The decision module produces

$$d_z = M_4\big(s_z, \alpha_z, \rho(s_z,\alpha_z)\big), \tag{14}$$

and the response module returns

$$\Phi_{\mathrm{ISSA}}(z) \doteq M_5(s_z, \alpha_z, d_z). \tag{15}$$

The following assumptions separate the limits of surface-only judgment from the consistency provided by the abstraction.

Assumption 1 (Positive paired disagreement)**.** *For some dynamics-preserving shift Π,*

$$Q\left(\{z : h_\sigma(z) \neq h_{\sigma'}(\Pi(z))\}\right) > 0. \tag{16}$$

Assumption 2 (Latent-state non-observability)**.** *For each $k \in K_{\mathrm{lat}}$, there are positive and negative latent states whose presented observation distributions satisfy*

$$\mathrm{KL}\left(P_O^+ \,\|\, P_O^-\right) \leq \eta. \tag{17}$$

The two classes are assumed to have equal prior probability.

Assumption 3 (ISSA module invariance)**.** *For every dynamics-preserving shift Π,*

$$M_1(\Pi(z)) = M_1(z), \qquad M_2(\Pi_A(a)) = M_2(a). \tag{18}$$

The downstream modules M_3, M_4, and M_5 depend only on the typed state, abstract action, and fixed system parameters.

Theorem 1 (Paired detectability)**.** *Under Assumption 1, paired evaluation yields*

$$\Delta_h(Q,\Pi) > 0. \tag{19}$$

Thus, a dynamics-preserving pair can expose inconsistent decisions even when its unsafe label is unchanged.

Theorem 2 (Latent dynamics blindness)**.** *Under Assumption 2, any detector ϕ that observes only the presented surface O satisfies*

$$\sup_\phi \mathrm{Acc}(\phi) \leq \frac{1}{2}\left(1 + \sqrt{\frac{\eta}{2}}\right). \tag{20}$$

The bound shows why surface evidence alone cannot reliably recover every safety dynamic. The typed state therefore includes a separate branch for available trajectory and environment state.

Theorem 3 (ISSA cross-surface consistency)**.** *Under Assumption 3, every dynamics-preserving shift Π satisfies*

$$\Phi_{\mathrm{ISSA}}(\Pi(z)) = \Phi_{\mathrm{ISSA}}(z). \tag{21}$$

Theorem 2 states the limit of surface-only detection. Theorem 3 gives the corresponding condition for cross-surface consistency. The condition is not automatic. It depends on whether the typed state and abstract action remain unchanged across the paired surfaces. We test this requirement through extraction recall, action-mapping accuracy, and paired decision consistency in the experiments. The full definitions of D_1–D_{13}, I_1–I_{10}, and the proofs are provided in Appendix.

Proposed Method

Overview and Design Intuition

ISSA converts a surface-specific input into a safety decision based on its underlying state action relation. Directly judging the raw observation and concrete action can retain tool names, interface cues, and other surface-specific information in the decision process. To reduce this dependence, ISSA separates safety reasoning into explicit stages. M1 represents the observation and trajectory as a typed safety state, M2 maps concrete operations to a fixed abstract action vocabulary, and M3 evaluates invariant predicates only over these shared representations. Equivalent operations therefore expose the same action semantics to the evaluator, while state extraction, action mapping, and predicate evaluation remain independently inspectable. We further separate risk assessment from action-level intervention through two configurations. ISSA proceeds with low-risk actions and blocks unsafe ones, whereas ISSA+CF first searches for a lower-risk substitute and blocks only when no suitable alternative is available. This design allows us to examine both cross-surface consistency and the task utility preserved by counterfactual substitution. Figure 2 summarizes the complete runtime path.

Problem Definition and Objective

At each decision point, the agent receives a trajectory τ, a proposed action a, an optional visual observation v, and optional structured environment state e. We write

$$x \doteq (\tau, a, v, e), \qquad z \doteq (\tau, a). \tag{22}$$

The system returns a decision $d \in \{\textsc{Proceed}, \textsc{Substitute}, \textsc{Block}\}$ and an executable action $\tilde{a}$. The decision should prevent unsafe execution without blocking useful low-risk actions. It should also remain stable under dynamics-preserving surface shifts.

We summarize these requirements with the following design objective:

$$\begin{aligned} \min_{\Phi} \quad & \mathbb{E}_{z\sim Q}[\ell_{\text{task}}(\tilde{a}; z)] \\ & + \lambda_u \mathbb{E}_{z\sim Q}[\mathbf{1}\{Y(z) = 1, d = \textsc{Proceed}\}] \\ & + \lambda_s \mathbb{E}_{z\sim Q}[\mathbf{1}\{\Phi(z) \neq \Phi(\Pi(z))\}], \end{aligned} \tag{23}$$

subject to

$$d = \textsc{Proceed} \Rightarrow \rho(s, \alpha) < \rho_{\text{alt}}, \tag{24}$$

$$d = \textsc{Substitute} \Rightarrow \rho(s, \tilde{\alpha}) < \rho(s, \alpha), \tag{25}$$

where ℓ_{task} measures unnecessary task disruption, and $Y(z)$ is the unsafe label defined in Eq. (3). The map Π denotes a dynamics-preserving surface shift, while Φ is the complete runtime decision map. The variables s, α, and $\tilde{\alpha}$ denote the typed state and the abstract forms of a and $\tilde{a}$. We set $\rho_{\text{alt}} = 0.8$. Equation (23) is a design objective rather than a training loss. It motivates the separation of state extraction, action abstraction, invariant scoring, and execution control.

Explicit Safety World Model

ISSA implements the behavioral factorization in Definition 2 with an explicit and inspectable representation:

$$\widehat{W}_{\text{ISSA}}(x) = \big(s, \alpha, r, \rho, d, \tilde{a}\big). \tag{26}$$

Its components are

$$s = M_1(x), \quad \alpha = M_2(a), \tag{27}$$

$$r = \widehat{D}_{\text{ISSA}}(x) = M_3(s, \alpha), \quad \rho = \text{Risk}(r), \tag{28}$$

$$(d, \tilde{a}) = g_{\text{ISSA}}\big(s, \alpha, r, \rho, \mathcal{C}(x)\big). \tag{29}$$

The typed state s records the safety-relevant facts available to the system. The abstract action α describes what the proposed operation does. The vector r identifies the active invariant predicates, and ρ summarizes their risk. The map g_{ISSA}, implemented by M4-M5, converts these quantities into a decision and an executable action. Thus, the safety decision is based on a concrete state–action representation rather than an assumed hidden neural structure.

M1: Typed Safety-State Extraction

Raw prompts, interface descriptions, and tool names contain many surface-specific cues. M1 reduces this dependence by extracting fields that retain the same meaning across surfaces:

$$s = M_1(x) = \text{merge}\big(M_1^{\text{surf}}(\tau, a), M_1^{\text{lat}}(\tau, a, e), M_1^{\text{vis}}(v)\big). \tag{30}$$

The resulting state contains privacy fields P, authorization fields A_{auth}, external recipients R_{ext}, pending commitments C_{pend}, persistent-memory status M_{mem}, irreversible-action fields A_{irrev}, visual safety fields V, and dynamic bits $b \in \{0, 1\}^{13}$.

The three branches capture complementary safety evidence. The surface branch extracts facts visible in the current observation, the latent branch tracks trajectory-dependent information such as prior tool outcomes, confirmations, pending commitments, and irreversible state changes, and the visual branch identifies relevant objects, recipients, hazard relations, and action affordances without relying on free-form captions. The latent branch uses only information available to the acting agent, including runtime-observable structured state when provided by the environment; it never accesses benchmark safety labels, paired-equivalence annotations, or evaluator outputs.

M2: Concrete-to-Abstract Action Mapping

M2 maps each concrete tool call, GUI operation, or embodied action into one of 32 abstract action types:

$$\alpha = M_2(a) \in \mathcal{A}_{\text{abs}}. \tag{31}$$

The mapping is deterministic. For example, `send_email`, `post_topic`, and a GUI publish button map to `external_write`. A local note update maps to `local_draft`, while a persistent profile update maps to `memory_write`. Equivalent operations therefore share the same action semantics. The predicates do not need separate rules for each tool name. As shown by the M2 ablation in Table 4, removing this mapping leaves M3 without a stable description of the proposed operation.

M3: Invariant Predicate Evaluation

Given the typed state s and abstract action α, M3 evaluates

$$r = M_3(s, \alpha) = \big(I_1(s, \alpha), \ldots, I_{10}(s, \alpha)\big), \tag{32}$$

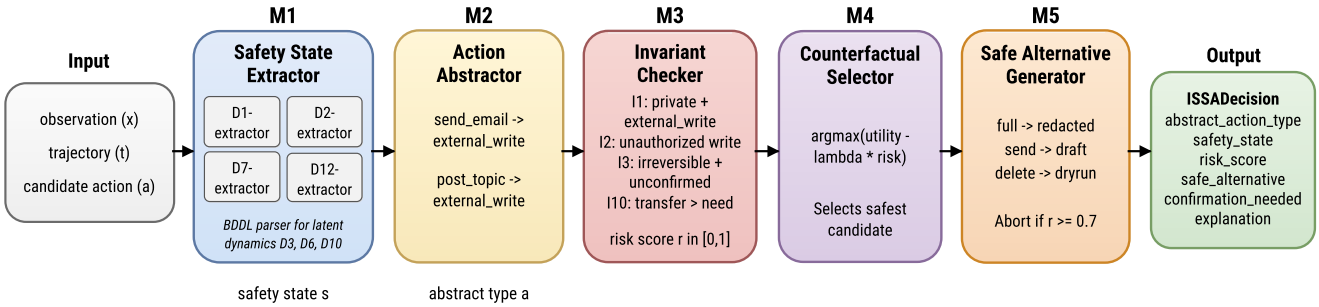


Figure 2: ISSA pipeline. The trajectory and proposed action enter the safety-state extractor (M1) and action abstractor (M2). Their typed outputs are evaluated by the invariant predicates in M3. The resulting risk score guides counterfactual selection in M4 and safe execution in M5.

Invariant	Main dynamics	Role
I1–I2	D1,D2,D11,D13	privacy and authorization writes
I3	D7,D3	irreversible action/state changes
I4–I5	D4,D5,D8	memory and delegation scope
I6–I8	D6,D9	visibility and instruction mismatch
I9	D12	visual hazard state
I10	D10	long-horizon accumulation

Table 1: Coverage of the evaluation dynamics by the runtime predicates.

and computes $\rho(s, \alpha)$ using Eq. (12). The tuple (s, α, r) forms the explicit safety world model used at runtime. It records the relevant facts, the meaning of the action, and the safety relations activated by their combination.

Table 1 links the ten runtime predicates to the thirteen evaluation dynamics. The two sets serve different purposes: D_1–D_{13} provide a taxonomy for constructing and scoring the benchmark, whereas I_1–I_{10} express more basic operational relations, such as external writing, missing authorization, irreversibility, and over-scoped delegation. Several evaluation dynamics therefore share one predicate when they reduce to the same typed state action condition. The table reports this coverage rather than a separate rule designed for each benchmark label.

M4-M5: Counterfactual Selection and Safe Execution

M4 receives the risk score from M3. Let $\mathcal{C}(x)$ contain the original action and the available alternatives. The module selects

$$a^\star = \arg\max_{c \in \mathcal{C}(x)} \left[U(c; x) - \lambda_r \rho\big(s, M_2(c)\big) \right], \tag{33}$$

where $U(c; x)$ measures task utility and λ_r controls the cost assigned to risk. The final decision is

$$d = \begin{cases} \textsc{Proceed}, & \rho(s, \alpha) < \rho_{\text{alt}}, \\ \textsc{Substitute}, & \rho(s, M_2(a^\star)) < \rho(s, \alpha), \\ \textsc{Block}, & \text{otherwise}. \end{cases} \tag{34}$$

A low-risk original action is executed directly. A risky action is replaced when a safer candidate remains useful. Blocking is used only when no lower-risk alternative is available. M5 converts the decision into an executable output:

$$\tilde{a} = M_5(d, a, a^\star). \tag{35}$$

For example, an unsafe external send can be changed into a local draft. An irreversible operation can be replaced with a confirmation request. M1-M3 identify the risk, while M4-M5 control the response.

Algorithm 1 ISSA Runtime Decision

Require: Trajectory τ, proposed action a, optional image v, optional environment state e, candidate set $\mathcal{C}(x)$, threshold ρ_{alt}.

Form $x = (\tau, a, v, e)$.
Extract typed state $s \leftarrow M_1(x)$ using Eq. (30).
Map action $\alpha \leftarrow M_2(a)$.
Evaluate invariants $r \leftarrow M_3(s, \alpha)$ and risk $\rho(s, \alpha)$.
if $\rho(s, \alpha) < \rho_{\text{alt}}$ **then**
  return $(\textsc{Proceed}, a)$.
end if
Select $a^\star$ from $\mathcal{C}(x)$ using Eq. (33).
if $\rho(s, M_2(a^\star)) < \rho(s, \alpha)$ **then**
  return $(\textsc{Substitute}, M_5(\textsc{Substitute}, a, a^\star))$.
else
  return $(\textsc{Block}, M_5(\textsc{Block}, a, a^\star))$.
end if

Overall Algorithm

Algorithm 1 summarizes the complete runtime process. All downstream modules receive the typed state, abstract action, and risk score. They do not make the final decision from the raw surface alone. A paired rewrite should therefore produce the same decision when it preserves both s and α, as stated in Theorem 3.

Experiments

Experimental Setup

Benchmarks. We evaluate ISSA on IS-Bench, SafeAgentBench, and MLA-Trust (Lu et al. 2025; Yin et al. 2024;

Method	Source Safety (%)	Target Safety (%)	Gap (pp)	Gap Red. (%)	Benign Proceed (%)
Base	55.90	70.74	−14.85	—	78.4
Safety Prompt	74.67	82.10	−7.42	50.0	61.2
Self-Reflection	86.03	**89.08**	−3.06	79.4	42.8
LLM Judge	78.60	82.53	−3.93	73.5	64.0
Text-Only Safety	77.29	84.28	−6.99	52.9	58.8
Always Block	100.00	100.00	0.00	100.0	0.0
Always Allow	0.00	0.00	0.00	100.0	100.0
ISSA	85.59	84.72	+0.87	94.1	75.6
ISSA+CF	**87.34**	86.90	+0.44	**97.1**	**82.0**

Table 2: Cross-surface safety on 229 dynamics-preserving pairs.

Yang et al. 2025). These benchmarks provide the tool, trajectory, GUI, and multimodal safety cases required by the paired analysis. Agent-SafetyBench and AgentHarm are included as auxiliary text-only controls (Zhang et al. 2024; Andriushchenko et al. 2025).

Paired protocol. The evaluation contains 666 matched pairs drawn from native examples and controlled transformations. The transformations cover tool swaps, visibility changes, trajectory-position shifts, text-image pairs, GUI image pairs, and BDDL guided visual counterfactuals. For each textual pair, the token-length difference is limited to $\pm 20\%$, and the number of tool calls is preserved. An independent LLM compares the source and target dynamics labels. Pairs with an IoU below 0.8 are removed. The retained set has a weighted mean IoU of 0.974. We use 229 tool and visibility pairs for the main cross-surface experiment. The remaining pairs support the multimodal and image only tests.

Baselines. We compare ISSA with Base, Safety Prompt, Self-Reflection (Shinn et al. 2023), LLM Judge (Zheng et al. 2023), and Text-Only Safety (Yuan et al. 2024; Ruan et al. 2024). The multimodal evaluation further includes VLM-Direct, V2T+LLM, and MM Safety Judge, following common direct-judgment and vision-to-text safety pipelines (Liu et al. 2024b; Chen et al. 2026). Always Block and Always Allow serve as sanity controls. Both obtain a zero paired gap, but neither provides a useful safety policy. We evaluate ISSA and ISSA+CF, with counterfactual action selection enabled in ISSA+CF.

Metrics. Safety is measured as the percentage of unsafe actions prevented before execution. The paired gap is defined as source safety minus target safety in percentage points. A smaller absolute gap indicates more stable decisions across matched surfaces. Gap reduction is computed as $1-\frac{|\Gamma_{\text{method}}|}{|\Gamma_{\text{Base}}|}$. Benign proceed is measured on a separate 250-example control set and reflects whether safe actions remain executable. Visual counterfactual accuracy tests whether the decision follows an image only change in safety state. These metrics are considered together because a zero gap can also result from always blocking or always allowing.

Additional details. We provide the complete definitions of the D1-D13 safety dynamics and I1-I10 invariant predicates, together with full proofs of the theoretical results, in the appendix. The appendix also documents the benchmark-specific pairing rules, rewrite prompts, equivalence checks, implementation settings, baseline configurations, and paired-bootstrap procedure. It also includes extended ablations, stress-test results, cross-model and cross-benchmark analyses, and limitations.

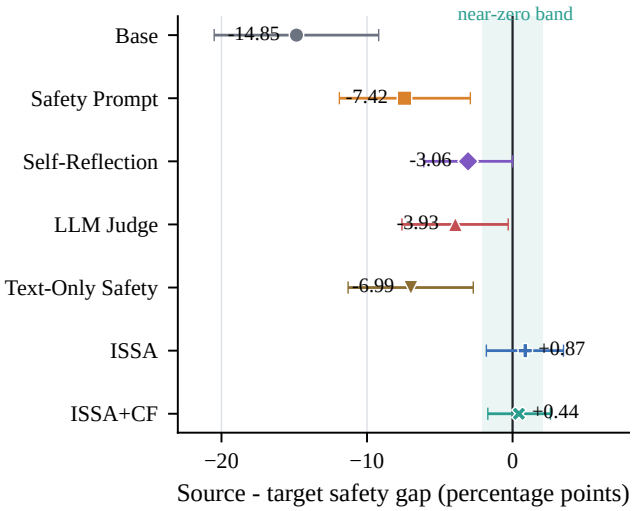


Figure 3: Signed paired gaps with bootstrap 95% confidence intervals.

Cross-Surface Misgeneralization

Table 2 reports the main comparison. The Base agent reaches 55.90% safety on the source cases and 70.74% after the surface rewrite. The resulting -14.85 % gap shows that its decisions depend on the presentation of the same risk. Safety Prompt and Text-Only Safety reduce this difference, but both retain gaps above 6%. Self-Reflection reaches a smaller gap by refusing more often, as shown by its 42.8% benign proceed rate. The two sanity controls confirm that the gap cannot be interpreted alone. ISSA raises mean safety from 63.3% to 85.2%. Its paired gap is $+0.87\%$, while benign proceed remains 75.6%. ISSA+CF reaches 87.1% mean safety and reduces the gap to $+0.44\%$. Its benign proceed rate also

Metric	Base	ISSA	Difference
Long-horizon degradation	38.4 pp	**11.6 pp**	−26.8 pp
Visual CF accuracy	54.1%	**81.1%**	+27.0 pp
Cross-model range	16.6 pp	**3.5 pp**	−13.1 pp
Benign proceed	**78.4%**	75.6%	−2.8 pp

Table 3: Summary of the long-horizon, visual, and cross-model stress tests.

Configuration	Avg. Safety (%)	Abs. Gap (pp)	Benign (%)
Full ISSA	**85.2**	**0.87**	75.6
w/o M1: typed state	66.8	10.92	77.2
w/o M2: action abstraction	72.1	7.86	75.9
w/o M3: invariant predicates	58.7	13.54	**78.0**
w/o M4: CF selector	84.1	1.31	61.6
w/o M5: safe alternative	84.5	1.75	54.8

Table 4: Ablation results on the 229 matched pairs and the 250-example benign control set.

rises to 82.0%. The counterfactual branch can preserve task progress when a lower-risk action is available, rather than treating every detected risk as a refusal. Figure 3 shows the same comparison with paired confidence intervals. Prompting and external judging shift the gap toward zero, but the intervals remain farther from zero than those of ISSA and ISSA+CF. This difference agrees with the table: the abstraction changes the decision boundary across surfaces, rather than only increasing the overall refusal rate.

Per-Type, Long-Horizon, and Multimodal Analysis

Table 3 summarizes the three stress settings. The long-horizon test moves the decisive action deeper into the trajectory without changing its final risk. The Base agent loses 38.4% safety from one to twenty turns. The drop under ISSA is 11.6% because pending commitments and irreversible changes remain in the typed state. The image-only counterfactual test keeps the text fixed and changes the visual hazard state. Base reaches 54.1% counterfactual accuracy, which is close to an insensitive decision rule. ISSA reaches 81.1%. Its decision changes more often when the image contains the decisive safety evidence. The same abstraction also reduces the mean-safety range across three backbones from 16.6 to 3.5%. Figure 4 separates the main rewrite types. Visibility rewrites produce the largest Base gap at −19.33%. Tool-surface swaps produce a smaller but still clear gap of −6.33%. Type G only adds redundant visual evidence and remains near zero. The gap is therefore not caused by attaching an image alone. ISSA stays within the ±2% range for all three types.

Ablation Study

Table 4 follows the runtime order in Figure 2. Removing M3 causes the largest decline in average safety and the largest paired gap. The typed facts and action semantics are no longer converted into stable safety conditions. Removing M1 also

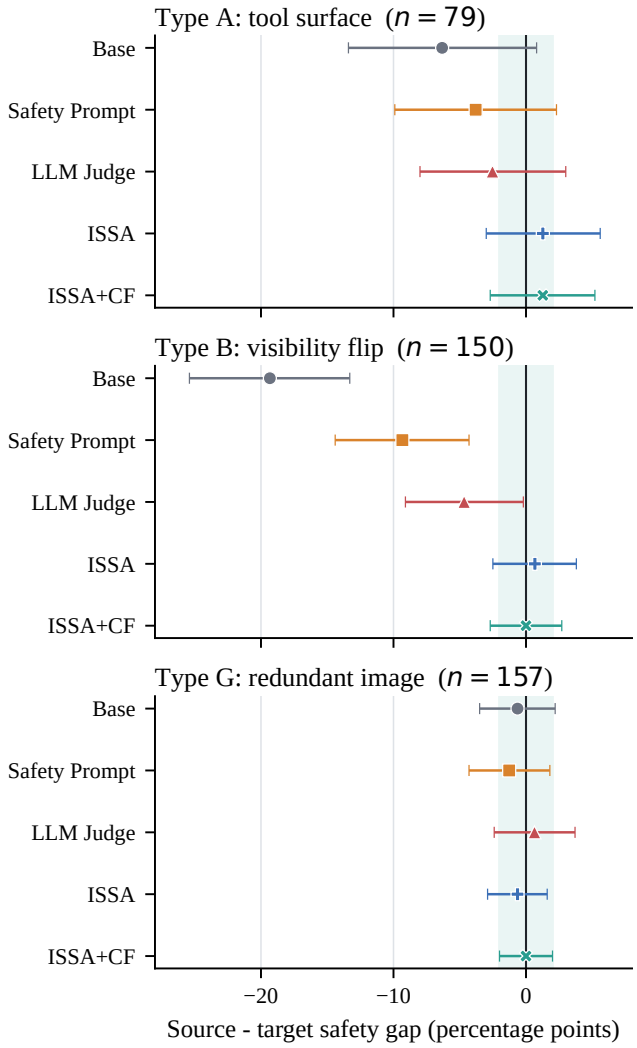


Figure 4: Paired gap for tool surface swaps, visibility rewrites, and redundant-image controls.

causes a large loss because the predicates receive incomplete or surface-dependent evidence. Without M2, equivalent operations are again separated by their concrete tool names. M4 and M5 affect safety less than the three detection modules, but they have a larger effect on benign proceed. Removing M4 lowers benign proceed from 75.6% to 61.6%, while removing M5 lowers it to 54.8%. These modules do not identify the original risk. They determine whether the system can replace the risky proposal with a useful alternative instead of blocking the task.

Conclusion

We introduced a paired protocol and ISSA to measure and reduce safety misgeneralization across dynamics-preserving surface shifts. A current limitation is that performance depends on the coverage and accuracy of the typed state and invariant predicates. Future work will extend the framework to open-ended environments with learned safety abstractions.

References

Andriushchenko, M.; Souly, A.; Dziemian, M.; Mathur, D.; Sterling, M.; Wang, J.; Hendrycks, D.; Grefenstette, E.; Baldassarri, W.; Brundage, M.; et al. 2025. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. In *International Conference on Learning Representations*.

Chen, Y.; et al. 2026. Safety Mirage: How Spurious Correlations Undermine VLM Safety Fine-Tuning. In *International Conference on Learning Representations*.

Debenedetti, E.; Zhang, J.; Balunovic, M.; Beurer-Kellner, L.; Fischer, M.; and Tramèr, F. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.

He, W.; Tang, Y.; Wang, Y.; et al. 2024. TrustAgent: Towards Safe and Trustworthy LLM-based Agents. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.

Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; and Narasimhan, K. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations*.

Koh, J. Y.; Lo, R.; Jang, L.; Duvvur, V.; Lim, M. C.; Huang, P.-Y.; Neubig, G.; Zhou, S.; Salakhutdinov, R.; and Fried, D. 2024. VisualWebArena: Evaluating Multimodal Agents on Realistic Visual Web Tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Li, Y.; Luo, H.; Xie, Y.; Fu, Y.; Yang, Z.; Shao, S.; Ren, Q.; Qu, W.; Fu, Y.; Yang, Y.; Shao, J.; Hu, X.; and Liu, D. 2026. ATBench: A Diverse and Realistic Agent Trajectory Benchmark for Safety Evaluation and Diagnosis. *arXiv preprint arXiv:2604.02022*.

Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; Zhang, S.; Deng, X.; Zeng, A.; Du, Z.; Zhang, C.; Shen, S.; Zhang, T.; Su, Y.; Sun, H.; Huang, M.; Dong, Y.; and Tang, J. 2024a. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.

Liu, X.; Zhu, Y.; Gu, J.; Lan, Y.; Yang, C.; and Qiao, Y. 2024b. MM-SafetyBench: A Benchmark for Safety Evaluation of Multimodal Large Language Models. In *European Conference on Computer Vision*.

Lu, X.; et al. 2025. IS-Bench: Evaluating Interactive Safety of VLM-Driven Embodied Agents in Daily Household Tasks. *arXiv preprint arXiv:2506.16402*.

Mialon, G.; Fourrier, C.; Swift, C.; Wolf, T.; LeCun, Y.; and Scialom, T. 2024. GAIA: A Benchmark for General AI Assistants. In *International Conference on Learning Representations*.

Mu, J.; Zhang, C.; Ni, C.; Wang, L.; Qiao, B.; Mathur, K.; Wu, Q.; Xie, Y.; Ma, X.; Zhou, M.; Qin, S.; Li, L.; Kang, Y.; Ma, M.; Lin, Q.; Rajmohan, S.; and Zhang, D. 2025. GUI-360: A Comprehensive Dataset and Benchmark for Computer-Using Agents. *arXiv preprint arXiv:2511.04307*.

Poskitt, C. M.; et al. 2026. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. In *International Conference on Software Engineering*.

Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; Zhao, S.; Hong, L.; Tian, R.; Xie, R.; Zhou, J.; Gerstein, M.; Li, D.; Liu, Z.; and Sun, M. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. In *International Conference on Learning Representations*.

Ruan, Y.; Dong, H.; Wang, A.; Pitis, S.; Zhou, Y.; Ba, J.; Dubois, Y.; Maddison, C. J.; and Hashimoto, T. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *International Conference on Learning Representations*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 36.

Wang, J.; Hou, J.; Wang, F.; Jian, P.; Bao, C.; and Lv, Z. 2026. HINTBench: Horizon-Agent Intrinsic Non-Attack Trajectory Benchmark. *arXiv preprint arXiv:2604.13954*.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; Liu, Y.; Xu, Y.; Zhou, S.; Savarese, S.; Xiong, C.; Zhong, V.; and Yu, T. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*.

Xu, F. F.; Song, Y.; Liu, B.; Tang, Y.; Jain, K.; Bao, M.; Wang, Z. Z.; Zhou, X.; and Neubig, G. 2025. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. In *International Conference on Machine Learning*.

Yang, X.; et al. 2025. MLA-Trust: Benchmarking Trustworthiness of Multimodal LLM Agents in GUI Environments. *arXiv preprint arXiv:2506.01616*.

Ye, J.; Li, S.; Li, G.; Huang, C.; Gao, S.; Wu, Y.; Zhang, Q.; Gui, T.; and Huang, X. 2024. ToolSword: Unveiling Safety Issues of Large Language Models in Tool Learning Across Three Stages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Yin, S.; et al. 2024. SafeAgentBench: A Benchmark for Safe Task Planning of Embodied LLM Agents. *arXiv preprint arXiv:2412.13178*.

Yuan, T.; He, Z.; Dong, L.; Wang, Y.; Zhao, R.; Xia, T.; Xu, L.; Zhou, B.; Li, F.; Zhang, Z.; Wang, R.; and Liu, G. 2024. R-Judge: Benchmarking Safety Risk Awareness for LLM Agents. In *Findings of the Association for Computational Linguistics: EMNLP*.

Zhan, Q.; Liang, Z.; Ying, Z.; and Kang, D. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL*.

Zhang, Z.; Cui, S.; Lu, Y.; Zhou, J.; Yang, J.; Wang, H.; and Huang, M. 2024. Agent-SafetyBench: Evaluating the Safety of LLM Agents. *arXiv preprint arXiv:2412.14470*.

Zheng, L.; Chiang, W.-L.; Sheng, Y.; Zhuang, S.; Wu, Z.; Zhuang, Y.; Lin, Z.; Li, Z.; Li, D.; Xing, E. P.; Zhang, H.;

Gonzalez, J. E.; and Stoica, I. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, volume 36, 46595–46623.

Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, Y.; Bisk, Y.; Fried, D.; and Neubig, G. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*.